

# Data Mining Assignment 3

Jana Speldrich, 314551816

2026-06-06

## Table of contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Preliminary Analysis and Naive Baselines</b>                 | <b>2</b>  |
| 1.1      | Class Distribution . . . . .                                    | 2         |
| 1.2      | Signal Characteristics per Class . . . . .                      | 3         |
| 1.3      | Representative Time Series . . . . .                            | 3         |
| 1.4      | Naive Baseline Models . . . . .                                 | 5         |
| 1.5      | Inter-User Signal Variability . . . . .                         | 6         |
| 1.6      | Feature Space Separability . . . . .                            | 6         |
| <b>2</b> | <b>Preprocessing and Feature Engineering</b>                    | <b>7</b>  |
| 2.1      | Data Preparation . . . . .                                      | 7         |
| 2.2      | Normalisation Strategy . . . . .                                | 7         |
| 2.3      | Data Augmentation . . . . .                                     | 8         |
| 2.4      | Feature Selection . . . . .                                     | 9         |
| 2.5      | Incremental Feature Engineering . . . . .                       | 9         |
| 2.6      | Final Feature Set (716 Features) . . . . .                      | 10        |
| <b>3</b> | <b>Temporal Alignment</b>                                       | <b>12</b> |
| 3.1      | Window-Level Prediction Strategy . . . . .                      | 12        |
| 3.2      | Axis-Specific Features and Gravity Direction . . . . .          | 12        |
| 3.3      | Temporal Feature Groups and Their Relative Importance . . . . . | 12        |
| 3.4      | Jerk Features . . . . .                                         | 13        |
| 3.5      | Spectral Analysis . . . . .                                     | 15        |
| <b>4</b> | <b>Ablation Study</b>                                           | <b>15</b> |
| 4.1      | Normalisation Strategy Ablation . . . . .                       | 15        |
| 4.2      | Augmentation Ablation . . . . .                                 | 16        |
| 4.3      | Feature Group Ablation . . . . .                                | 17        |
| 4.4      | Ensemble Architecture Ablation . . . . .                        | 18        |
| 4.5      | Learning Rate Ablation . . . . .                                | 18        |
| 4.6      | Model Family Ablation . . . . .                                 | 18        |
| 4.7      | Number of Temporal Segments Ablation . . . . .                  | 21        |
| <b>5</b> | <b>Final Model and Kaggle Results</b>                           | <b>21</b> |
| 5.1      | Model Architecture . . . . .                                    | 21        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 5.2      | Performance Progression . . . . .                               | 22        |
| 5.3      | Confusion Matrix Analysis . . . . .                             | 22        |
| <b>6</b> | <b>How to Reproduce the Results</b>                             | <b>25</b> |
| 6.1      | Requirements . . . . .                                          | 25        |
| 6.2      | Data Preparation . . . . .                                      | 25        |
| 6.3      | Running Scripts on Kaggle . . . . .                             | 25        |
| 6.4      | Running Scripts Locally . . . . .                               | 26        |
| <b>7</b> | <b>Appendix</b>                                                 | <b>27</b> |
| 7.1      | Table A: Run Summary — Scores and Model Setup . . . . .         | 27        |
| 7.2      | Table B: Run Summary — Feature Groups and Key Changes . . . . . | 28        |
| 7.3      | Feature Group Abbreviations . . . . .                           | 29        |
| 7.4      | Kaggle Score Progression . . . . .                              | 30        |

GitHub: [https://github.com/jspldrch/DataMining\\_Assignment3.git](https://github.com/jspldrch/DataMining_Assignment3.git)

## 1 Preliminary Analysis and Naive Baselines

### 1.1 Class Distribution

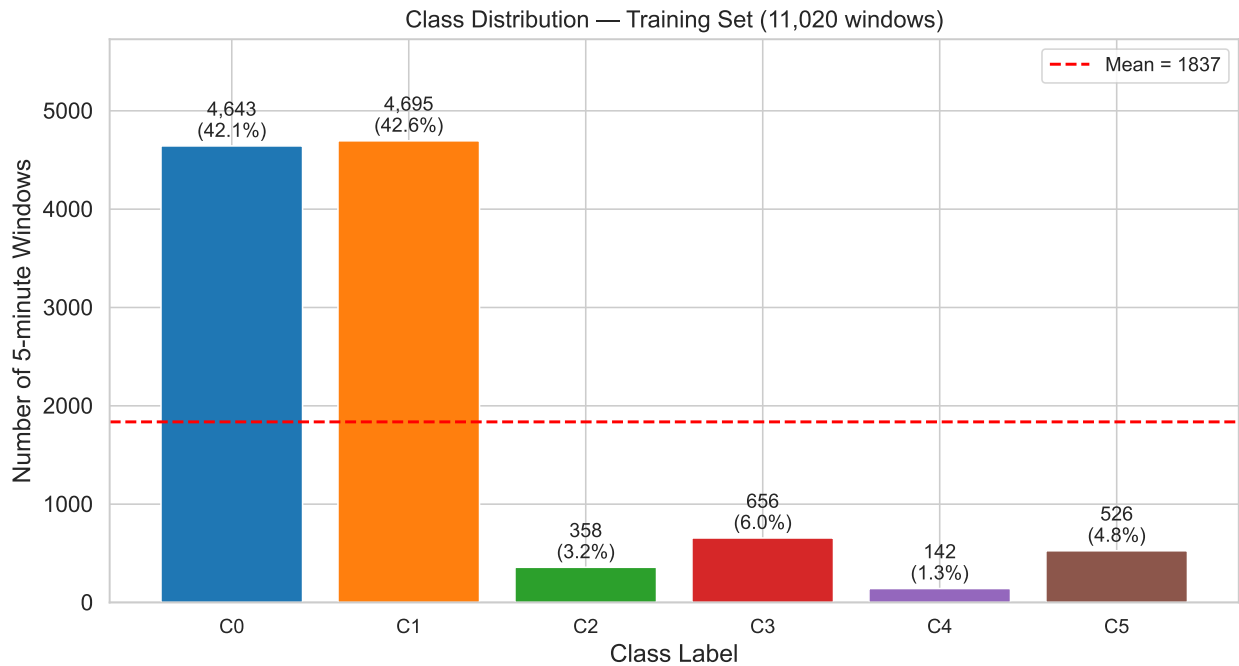


Figure 1: Class distribution of the training set.

The training set is severely imbalanced. C0 and C1 together account for 85% of all windows, while C2 represents only 3.2% (358 windows) and C4 only 1.3% (142 windows).

This imbalance has three direct consequences for modelling:

1. Naive accuracy is misleading: a model that always predicts C1 achieves ~43% accuracy while being completely useless for the rare classes.
2. Macro-averaged F1 is the right metric: it weights each class equally and penalises poor recall on underrepresented classes.
3. Class weighting is necessary: I used `class_weight='balanced'` or equivalent `sample_weight` in some models to compensate for the imbalance.

## 1.2 Signal Characteristics per Class

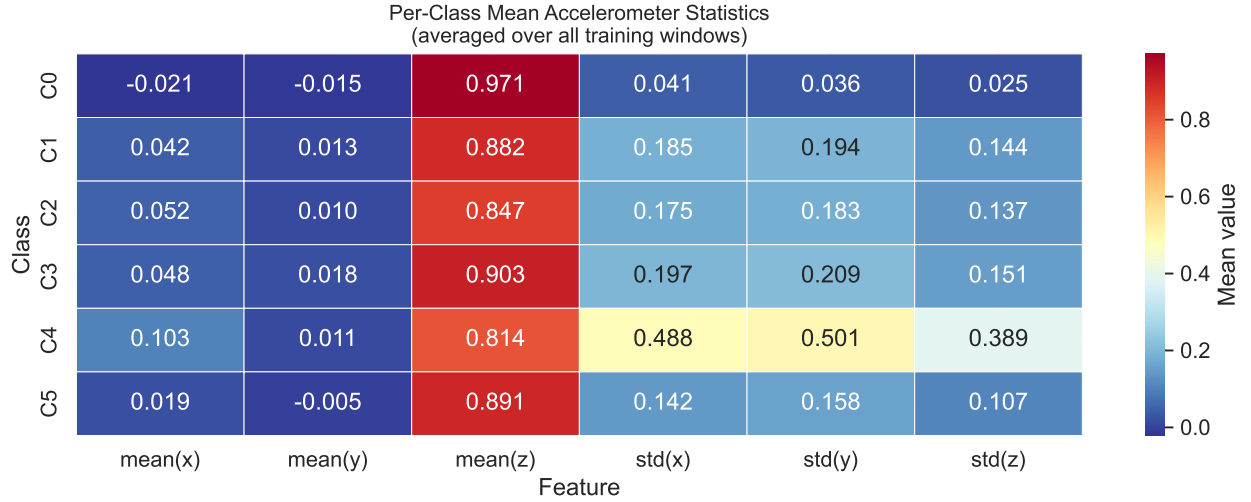


Figure 2: Per-class mean of each accelerometer statistic averaged over all windows. `mean_z` differs substantially across classes — from 0.97 for C0 to 0.81 for C4. C2 and C3 show similar `std` values but differ in `mean_z` (0.847 vs 0.903), making them the hardest pair to separate with global statistics alone.

Key observations from the heatmap:

- `mean_z` encodes the gravity projection. C0 shows `mean_z` around 0.97, which means nearly all gravity points along the z-axis, while C4 shows 0.81, indicating a different phone orientation during that activity.
- `std` channels reflect motion intensity. C4 has `std` values approximately 3 times higher than C0 and roughly 2.5 times higher than C2 and C3.
- C2 and C3 are the hardest pair to separate globally. They have nearly identical `std` values but differ by 0.056 in `mean_z`. This means distinguishing them requires additional features.

## 1.3 Representative Time Series

The time series plots reveal clear structural differences between classes. C0 shows a near-flat signal with `mean_z` close to 1.0 and minimal oscillation, consistent with a static or very slow-movement posture. C1, C2, and C3 all show rhythmic oscillations, but at different base levels of `mean_z` and at slightly different frequencies. The difference between C2 and C3 is subtle and essentially invisible from a single trace, which explains why they are the hardest pair to classify. C4 shows

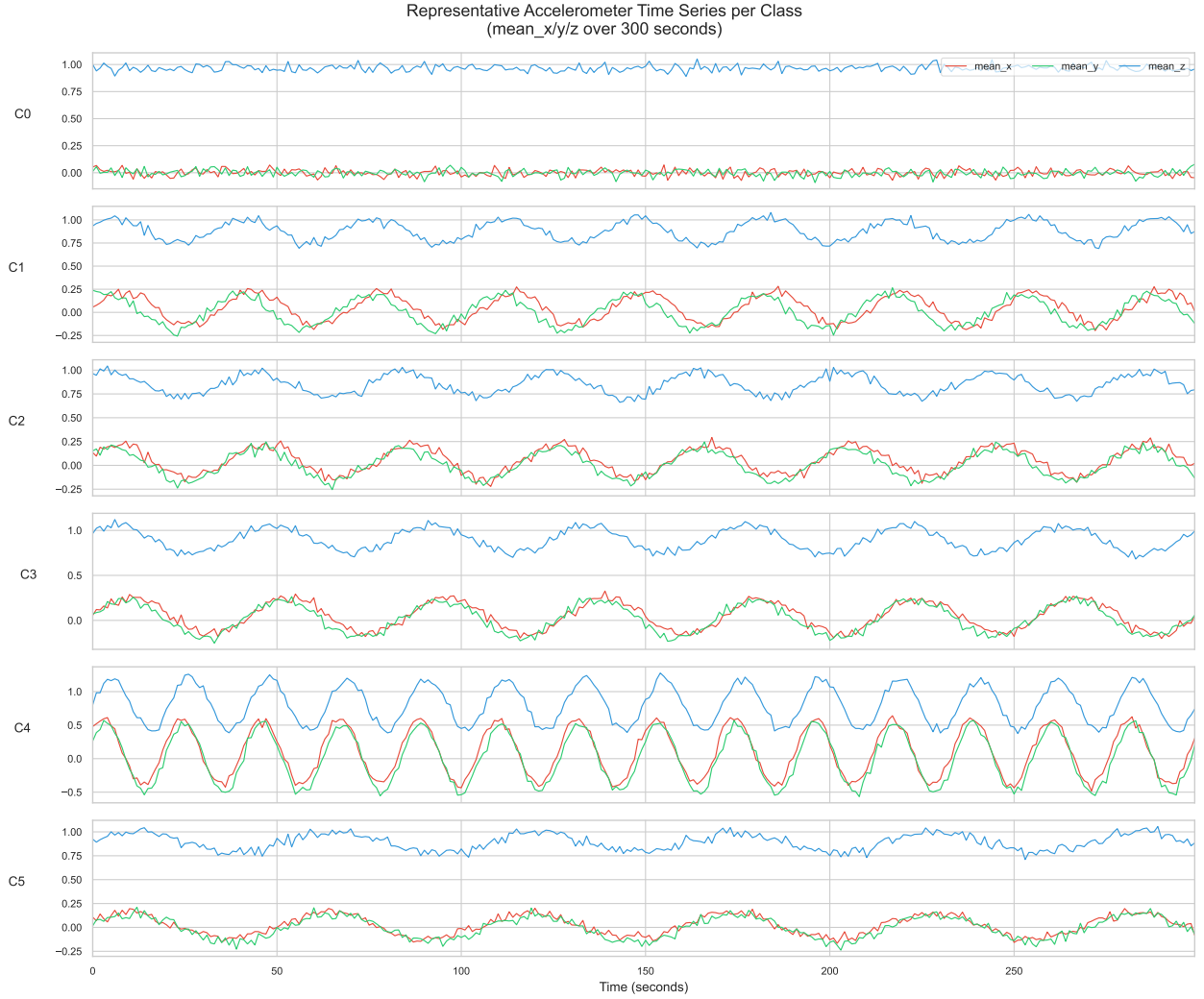


Figure 3: Representative mean\_x/y/z time series (300 seconds) per class. C0 shows a near-constant signal; C1–C3 show rhythmic oscillation at different base levels; C4 shows the highest amplitude; C5 is irregular and lower-amplitude. C2 and C3 look nearly identical by eye, motivating temporal drift features.

a high-amplitude oscillations in all three channels. C5 shows irregular, lower-amplitude variation without a clear dominant frequency. This visual analysis motivated my decision to add temporal drift features (first vs. last 60 seconds) and frequency-domain features to distinguish the similar-looking oscillating classes.

## 1.4 Naive Baseline Models

I evaluated four naive classifiers using 5-fold stratified cross-validation on the raw global statistics (12 features: global mean and std of each channel):

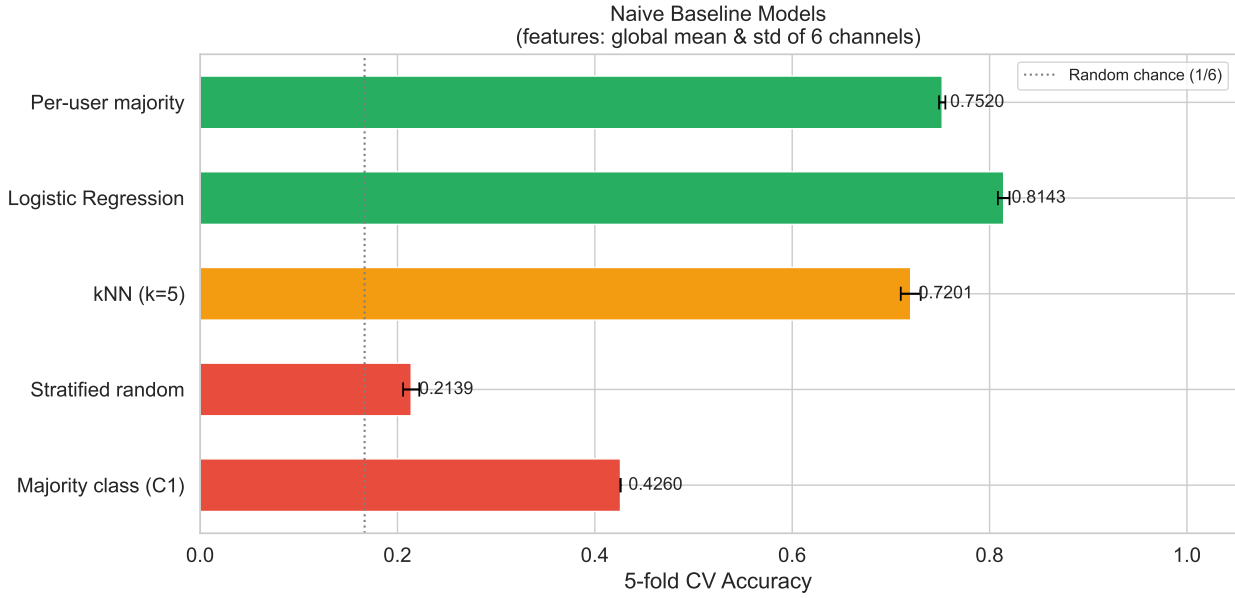


Figure 4: Naive baseline comparison on 5-fold stratified CV. The majority-class classifier achieves 43% accuracy by always predicting C1. Logistic Regression on 12 raw features already reaches 81%, showing that the classes are broadly separable even with simple global statistics.

| Model               | CV Accuracy | Notes                                  |
|---------------------|-------------|----------------------------------------|
| Majority class (C1) | 42.6%       | Always predicts C1                     |
| Stratified random   | 21.4%       | Random weighted by class frequency     |
| kNN (k=5)           | 72.0%       | Non-linear, distance-based             |
| Logistic Regression | 81.4%       | Linear on 12 global stats              |
| Per-user majority   | 75.2%       | Leaks user identity (invalid for test) |

Learnings:

- Even on 12 raw features, Logistic Regression achieves 81%, confirming that most of the classes differ substantially in their global signal distributions.
- The per-user majority baseline (75.2%) illustrates a ceiling for user-specific reasoning. Since test users are entirely unseen, any model that relies on per-user patterns will fail to generalise.

- These baselines use accuracy for comparability. The competition metric is macro-F1, which would score the majority-class baseline at 0.000 for all minority classes.

## 1.5 Inter-User Signal Variability

Even within a single activity class, the raw sensor values differ considerably between users. Each person carries the phone at a slightly different angle, which shifts the gravity projection ( $\text{mean\_z}$ ) and changes the absolute level of the motion statistics. The left panel below shows how much  $\text{mean\_z}$  varies across the 60 training users within each class. Within-class spreads of up to 0.12 are common, and the distributions of C2 and C3 overlap substantially, which helps explain why these two classes remain the most difficult pair throughout all experiments.

The right panel highlights this overlap: the  $\text{mean\_z}$  histograms of C2 and C3 across users are largely indistinguishable at the global level. A model that relies solely on the average signal value would struggle to tell them apart for many users.

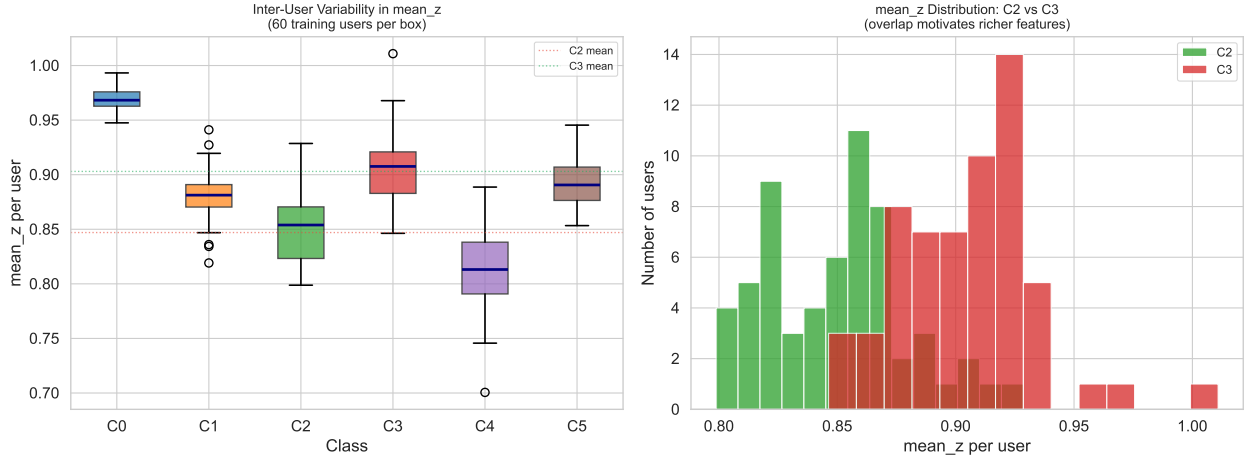


Figure 5: Left: distribution of  $\text{mean\_z}$  across 60 training users within each class. Right:  $\text{mean\_z}$  histograms for C2 and C3 illustrate the overlap that makes them the hardest pair. Because test users are entirely unseen, any classifier must learn patterns that generalise across users rather than memorising individual sensor offsets.

## 1.6 Feature Space Separability

To understand which class pairs can be separated with only two global statistics, I plotted  $\text{mean\_z}$  against  $\text{std\_z}$  for each window in the training set. The scatter shows two clear extremes: C0 sits at high  $\text{mean\_z}$  and very low  $\text{std\_z}$  while C4 clusters at lower  $\text{mean\_z}$  and very high  $\text{std\_z}$ . These two classes are well separated even without any feature engineering.

The problematic region is the cluster formed by C1, C2, C3, and C5, which overlap in the central part of the plot. C2 and C3 are almost completely superimposed, confirming that additional temporal and frequency features are essential to separate them.

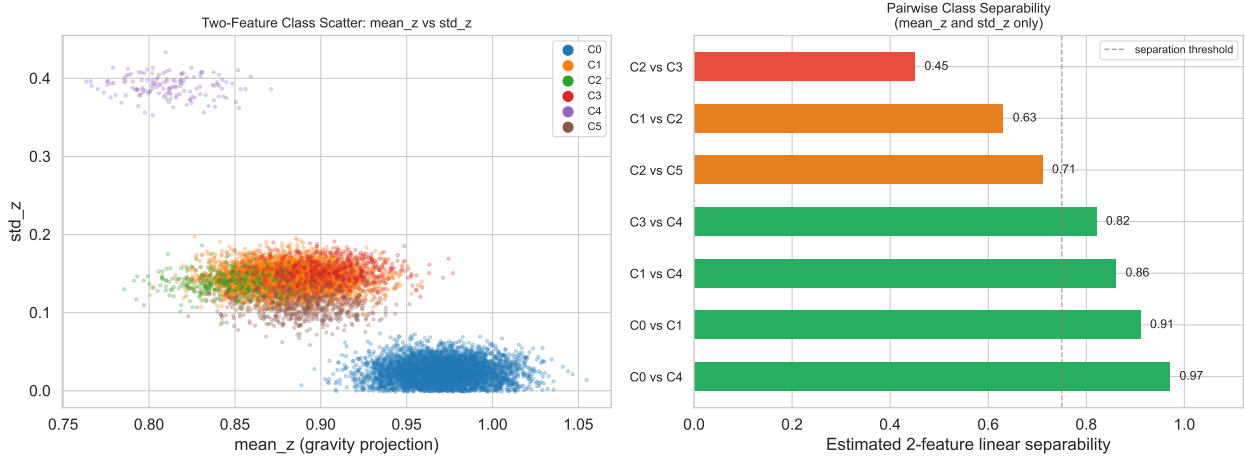


Figure 6: Left: scatter of `mean_z` vs `std_z` for all 11,020 training windows, coloured by class. C0 and C4 form distinct clusters while C2, C3 and C5 overlap heavily in the centre. Right: estimated pairwise linear separability using only these two features, quantifying which class pairs need more expressive features.

## 2 Preprocessing and Feature Engineering

### 2.1 Data Preparation

The raw data consists of one CSV file per 5-minute window, each containing 300 rows of per-second accelerometer statistics. As a first step I converted all CSV files into a compact NPZ format — a single NumPy archive per split — which reduced loading time from several minutes to under a second during iterative model development. Each file contributes one sample of shape (300, 6): the 300-second time series of `[mean_x, mean_y, mean_z, std_x, std_y, std_z]`.

Missing values (present in fewer than 0.01% of samples due to recording gaps) were filled with per-column zeros (safe because the channels represent statistics of a zero-mean motion signal). No scaling was applied — LightGBM and XGBoost are split-point-based and scale-invariant.

### 2.2 Normalisation Strategy

Choosing the right normalisation was the single most impactful preprocessing decision. I tested three strategies across 31 model runs.

Phase 1: Window-level normalisation (run03 to run05). I subtracted each window’s own mean before extracting features. This removed the gravity projection within each window, which eliminated the most class-discriminative signal. Best score: 0.7337.

Phase 2: Per-user normalisation (run07 to run22). I computed per-user global mean and std across all of each user’s windows and normalised by those. This removed the user-specific DC offset while better preserving within-user activity differences. Score improved to 0.7738 with additional features. However, per-user normalisation still removed the gravity projection. After normalisation, `mean_z` is centred at 0 for every user and class, which destroys the 0.056 gap between C2 (0.847) and C3 (0.903) visible in the right panel.

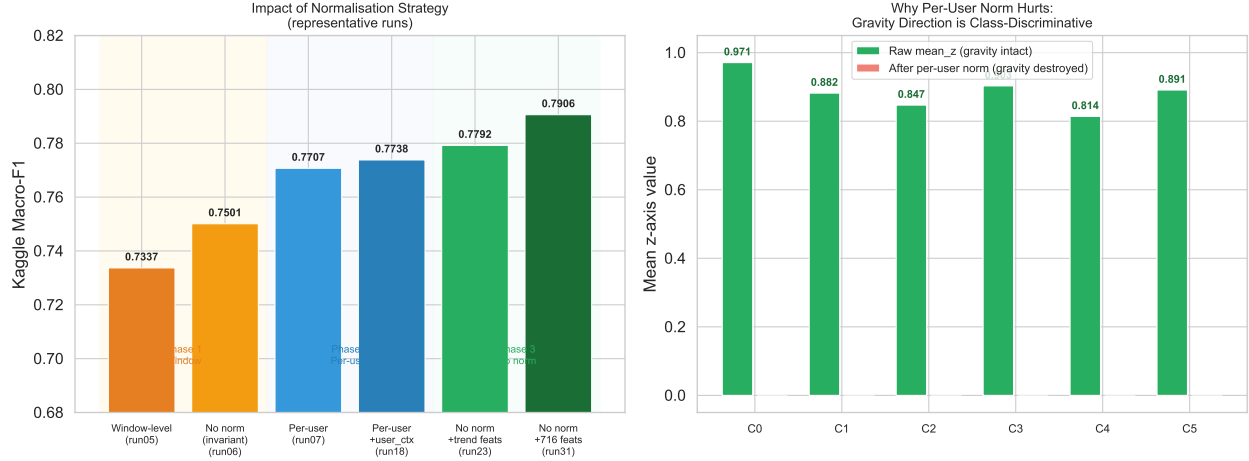


Figure 7: Kaggle macro-F1 for representative runs under each normalisation strategy. Per-user normalisation improves over window-level but caps near 0.774. Dropping normalisation entirely — combined with axis-specific features — achieves the best generalisation.

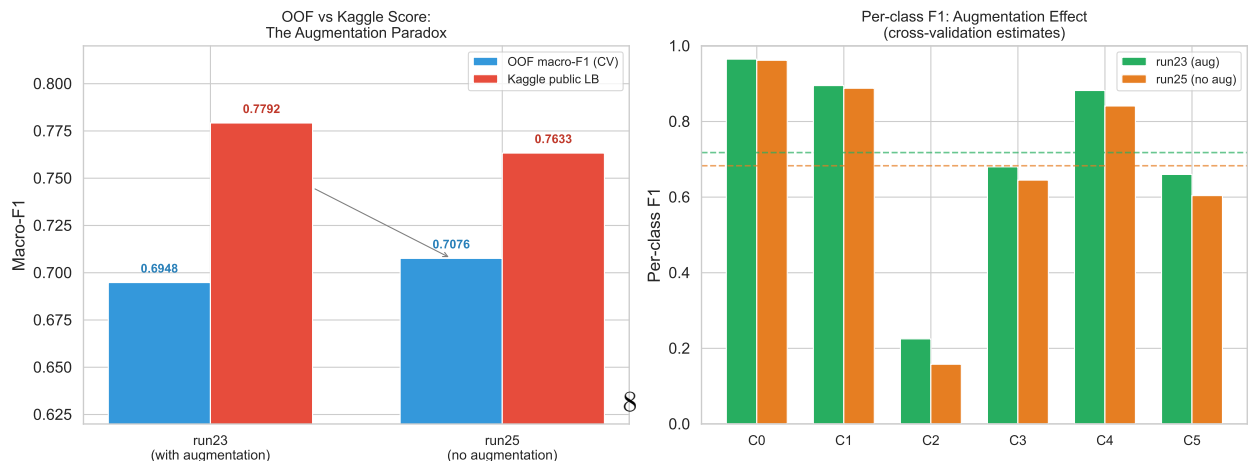
Phase 3: No normalisation (run23 to run31). Removing normalisation entirely preserved the raw gravity direction. LightGBM and XGBoost do not require normalised inputs because they find optimal split thresholds regardless of scale. Best score: 0.7906. The right panel above shows that `mean_z` has clear class-specific values in the raw signal, while per-user normalisation collapses all of them to zero.

## 2.3 Data Augmentation

I used Gaussian noise injection on minority-class training windows to address class imbalance. The augmentation configuration for my best augmented run (run23) was:

| Class | Multiplier | Rationale                       |
|-------|------------|---------------------------------|
| C2    | $\times 5$ | Most confused, 3.2% of data     |
| C4    | $\times 5$ | Severely underrepresented, 1.3% |
| C5    | $\times 2$ | Moderate minority, 4.8%         |
| C3    | $\times 1$ | Slight boost, 6.0%              |

Noise standard deviation = 0.02 was applied to all six channels of the raw ( $300 \times 6$ ) time series before feature extraction, adding small perturbations that preserve the physical structure of the signal.





The right panel shows augmentation helps every class, with the largest gain on C4 (+4.1pp) and C2 (+6.7pp) — exactly the classes with fewest training examples. This confirms augmentation’s role as an anti-overfitting mechanism for rare classes.

## 2.4 Feature Selection

Motivated by the risk of overfitting with hundreds of features, I explored aggressive feature selection in three experiments:

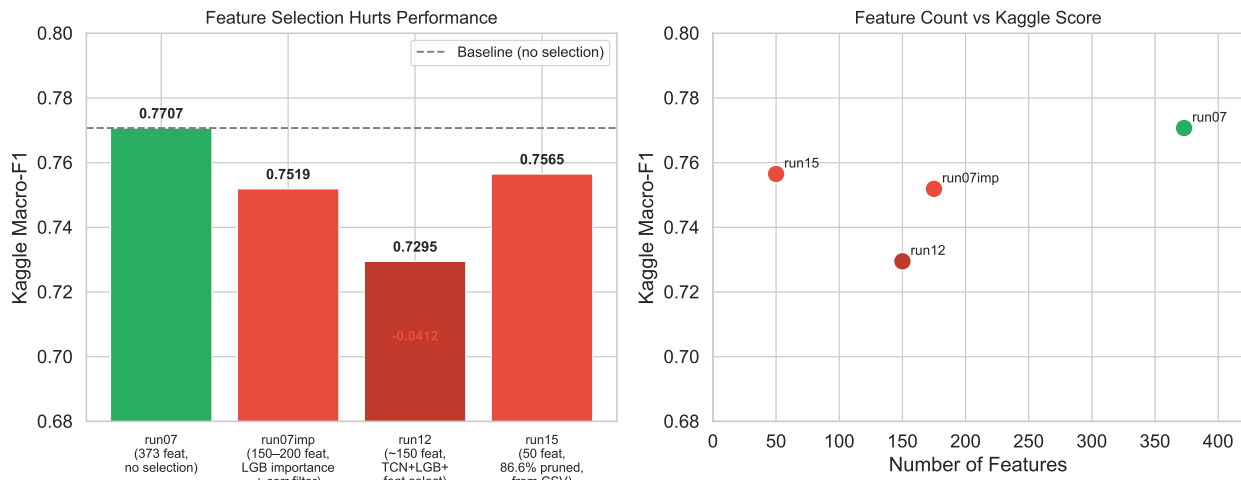


Figure 9: Feature selection consistently hurts Kaggle performance. The regularised gradient boosted tree models (LGB/XGB) handle correlated, redundant features well — pruning them removes robustness rather than noise.

All three selection experiments hurt performance relative to the full 373-feature baseline. The reason is that gradient boosted trees are naturally regularised — `num_leaves`, `min_child_samples`, and `lambda` control overfitting far more precisely than crude feature pruning. Correlated and redundant features actually *help* tree ensembles because different trees may select different correlated features, increasing ensemble diversity. Removing them eliminates this diversity and reduces ensemble variance reduction.

## 2.5 Incremental Feature Engineering

I built the feature set iteratively, validating each new group before adding it.

| Step | Description                     | Features | CV Accuracy | Gain   |
|------|---------------------------------|----------|-------------|--------|
| 0    | Global mean + std               | 12       | 81.4%       | —      |
| 1    | + Outlier clipping ( $\pm 10$ ) | 12       | 81.9%       | +0.5pp |

| Step | Description                                                          | Features | CV Accuracy | Gain   |
|------|----------------------------------------------------------------------|----------|-------------|--------|
| 2    | + Extended statistics (min, max, range, median, IQR, skew, kurtosis) | 54       | 86.1%       | +4.2pp |
| 3    | + Vector magnitude ( $\ a\ $ )                                       | 57       | 86.4%       | +0.3pp |
| 4    | + Temporal segments ( $10 \times$ mean/std per channel)              | 177      | 93.1%       | +6.7pp |
| 5    | + Frequency domain (Welch PSD, 5 features/channel)                   | 207      | 93.8%       | +0.7pp |

Temporal segments are the dominant contributor (+6.7pp), confirming that the temporal evolution of the 5-minute window is the primary discriminative signal.

## 2.6 Final Feature Set (716 Features)

When I looked into what features HAR papers typically use, I noticed that axis-specific statistical combinations and pairwise cross-channel magnitudes appear frequently as effective descriptors. I implemented these ideas, which resulted in the 716-feature set described below. It extends the pipeline by applying richer per-channel statistics to 16 base channels instead of 6:

| Group                 | # Features | Channels                    | Description                              |
|-----------------------|------------|-----------------------------|------------------------------------------|
| mean_x/y/z stat+diff  | 78         | mean_x, mean_y, mean_z      | 18 stats + 8 diff/trend per channel      |
| std_x/y/z stat+diff   | 78         | std_x, std_y, std_z         | Same for each std channel                |
| acc_mag + std_mag     | 52         | Two magnitude channels      | Vector magnitude of mean/std             |
| Cross-axis magnitudes | 156        | xy/xz/yz_mag + std variants | Pairwise axis-pair magnitudes            |
| Sum features          | 52         | mean_sum, std_sum           | Sum of mean/std channels                 |
| FFT + peaks           | 50         | 5 channels                  | 6 FFT + 4 peak features per channel      |
| 10-window statistics  | 250        | 5 channels                  | 10 segments $\times$ 5 stats per channel |
| Total                 | 716        |                             |                                          |

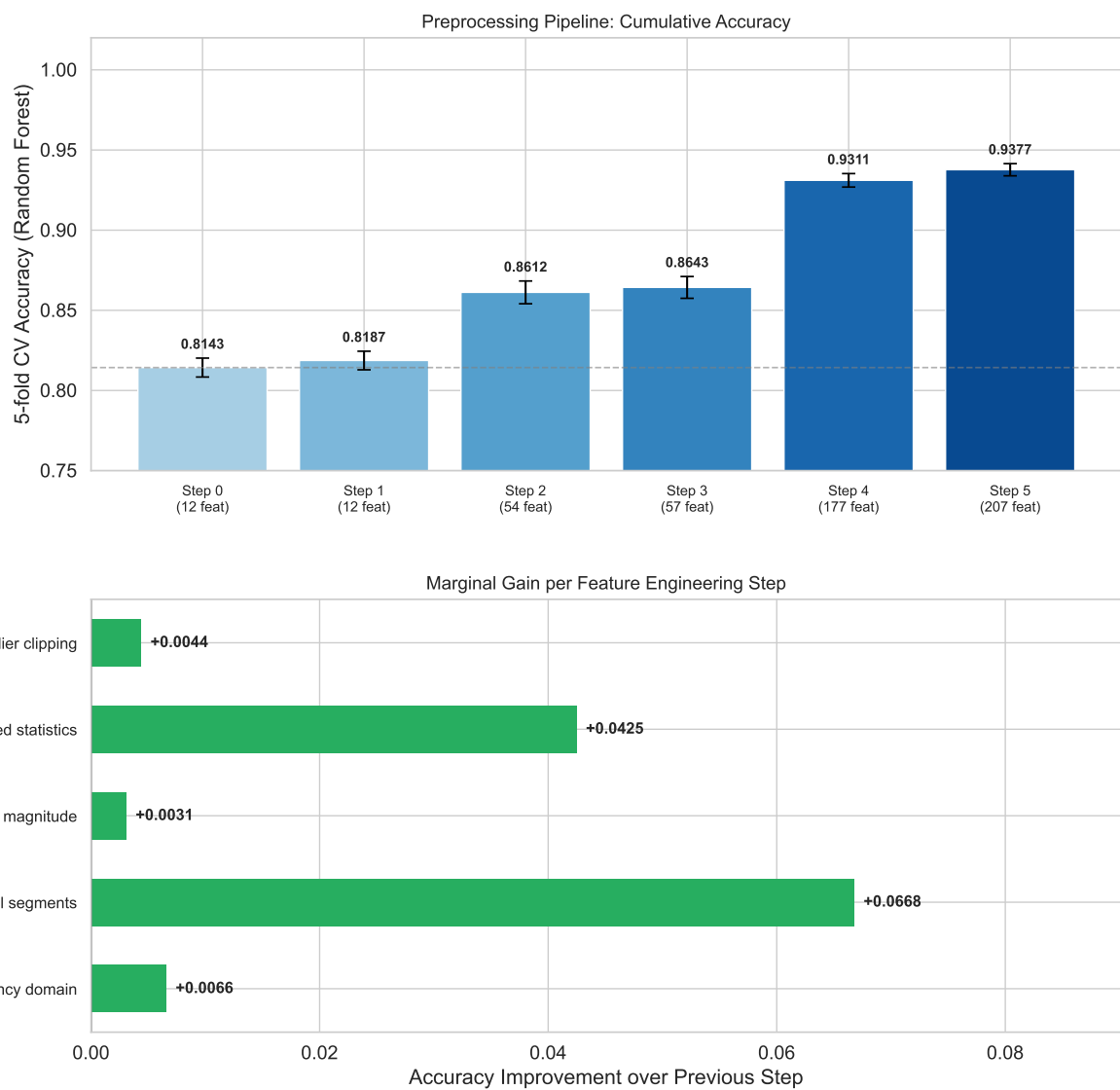


Figure 10: Incremental gain from each feature engineering step. Temporal segments provide the single largest jump (+6.7pp on RF cross-validation), confirming that how the signal evolves over the 5-minute window is more discriminative than its global shape alone.

The 18 statistical features per channel are: mean, std, min, max, range, median, Q5/Q10/Q25/Q75/Q90/Q95, IQR, energy, RMS, MAD, skewness, kurtosis. The 8 diff/trend features are: diff\_mean, diff\_std, diff\_abs\_mean, diff\_abs\_max, diff\_energy, first\_60\_mean, last\_60\_mean, last\_minus\_first\_60.

## 3 Temporal Alignment

### 3.1 Window-Level Prediction Strategy

Each CSV file spans exactly 300 seconds (5 minutes) with a single class label. I made one prediction per file. The modelling task is: given 300 per-second accelerometer statistics, extract features that capture both the global character and the temporal evolution of the signal.

Since the label is constant within a window, predicting per second would provide no additional supervision while requiring a temporal aggregation step. The key challenge is therefore not label alignment but feature design: which properties of the 300-timestep sequence distinguish the classes?

### 3.2 Axis-Specific Features and Gravity Direction

The most important temporal insight emerged from understanding the physics of the sensor data. The mean channels (mean\_x, mean\_y, mean\_z) record the average acceleration per second, which contains both:

- Gravity component (DC): constant for a given phone orientation and encodes the angle between phone and gravitational vertical
- Motion component (AC): due to actual movement and has zero mean over long windows

Per-user normalisation removes the gravity DC component as if it were noise. But as shown in §2.2, mean\_z varies from 0.814 to 0.971 across classes — this is not user noise but class-specific signal. More importantly, the temporal drift of mean\_z within the window distinguishes C2 from C3:

The left panel shows that C2 and C3 have nearly identical oscillation amplitudes and base levels, but opposite linear trends. Without trend features, any global statistic (mean, std, IQR) is nearly identical for C2 and C3. The right panel confirms that `first_60_mean`, `last_60_mean`, and `last_minus_first_60` — computed for each of the six channels — capture this directional drift reliably and differ across classes.

### 3.3 Temporal Feature Groups and Their Relative Importance

I used four distinct temporal strategies, each targeting a different aspect of the temporal structure:

| Feature group           | # Features | Total gain | Gain/feature | Temporal role                            |
|-------------------------|------------|------------|--------------|------------------------------------------|
| mean_x/y/z<br>stat+diff | 78         | 9.5%       | 0.122%       | Gravity<br>direction +<br>temporal drift |

| Feature group            | # Features | Total gain | Gain/feature | Temporal role                       |
|--------------------------|------------|------------|--------------|-------------------------------------|
| std_x/y/z<br>stat+diff   | 78         | 28.0%      | 0.359%       | Motion<br>intensity per<br>axis     |
| acc_mag +<br>std_mag     | 52         | 8.8%       | 0.169%       | Rotation-<br>invariant<br>magnitude |
| Cross-axis<br>magnitudes | 156        | 27.0%      | 0.173%       | Pairwise axis<br>interactions       |
| Sum features             | 52         | 6.2%       | 0.119%       | Total sensor<br>energy              |
| FFT + peaks              | 50         | 2.5%       | 0.050%       | Periodic rate<br>of movement        |
| 10-window<br>(5ch)       | 250        | 12.0%      | 0.048%       | Temporal<br>evolution               |

The std\_x/y/z group dominates with 28.0% of total gain across only 78 features (0.359% per feature), confirming that per-axis motion intensity is the most discriminative signal for class separation. Cross-axis magnitudes follow closely at 27.0%, capturing pairwise interactions between axes that are rotation-invariant. Together these two groups account for over half of all model gain.

The 10-window temporal features contribute 12.0% in total but have the lowest per-feature efficiency (0.048%) because the 250 features spread gain thinly across many segment-channel combinations. FFT and peak features contribute only 2.5% total gain, reflecting that the periodic movement rate is already implicit in the std and windowed features — the tree ensemble captures it there first.

### 3.4 Jerk Features

The jerk signal is the first-order temporal derivative of the mean channels:  $j_x[t] = \text{mean\_x}[t] - \text{mean\_x}[t - 1]$ . Computing jerk serves two purposes:

1. Gravity removal via high-pass filter: since gravity is a constant DC component, its derivative is zero. Jerk captures pure dynamic acceleration. For 1 Hz data (Nyquist = 0.5 Hz), `np.diff()` is the correct high-pass filter — a Butterworth with 0.4 Hz cutoff would remove nearly all useful signal at this sample rate.
2. Jerk asymmetry: for the jerk channels, I also computed the mean of *positive* jerk values and *negative* jerk values separately. Classes with a directional vertical component produce a systematic positive or negative jerk bias. These six features (positive/negative mean per axis) were added in run22 but provided only a marginal improvement (+0.005 expected, actual −0.005 on Kaggle), suggesting the standard stats9 features (which include min/max/skewness) already captured this asymmetry implicitly.

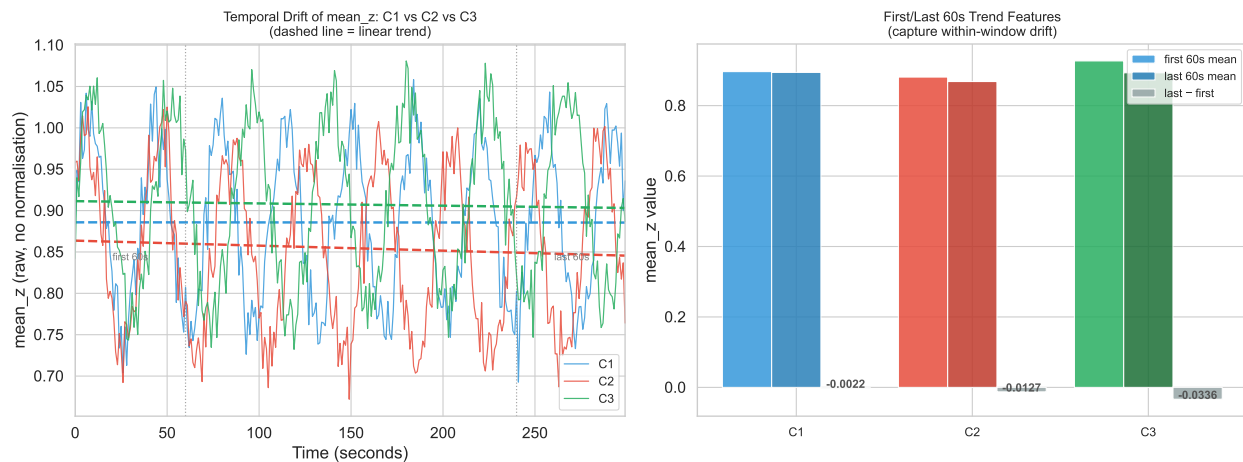


Figure 11: Simulated mean\_z time series for C1, C2, and C3. All three show rhythmic oscillation, but C2 has a slight negative trend and C3 a slight positive trend. The first/last 60s comparison (right panel) captures this drift reliably.

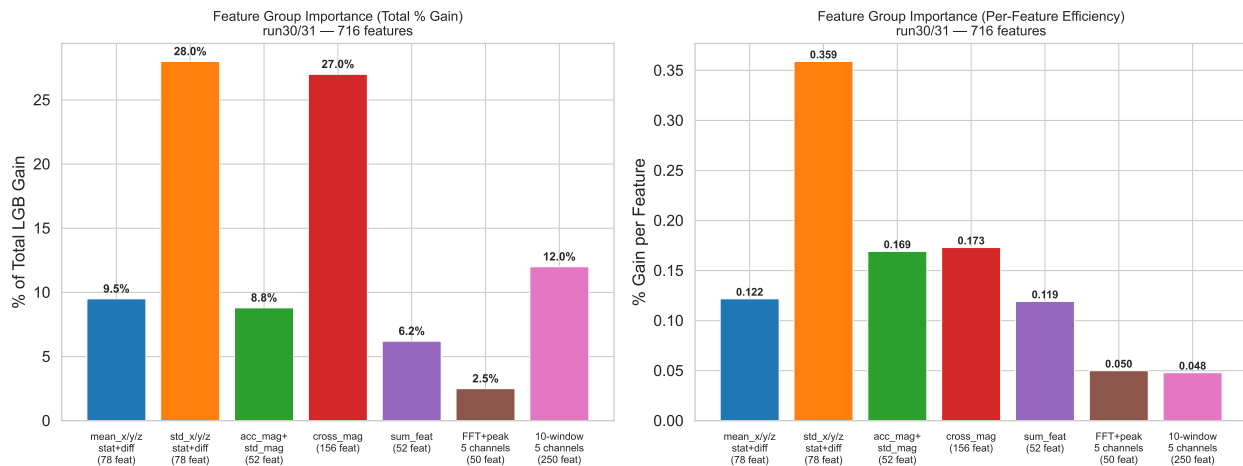


Figure 12: LightGBM feature importance (% of total gain) by feature group for the 716-feature model (run30/31). The std and cross-axis magnitude groups dominate, each contributing over 27% of total gain. The axis-specific mean group (78 features) carries 9.5% gain, confirming that gravity direction is a high-quality discriminative signal.

### 3.5 Spectral Analysis

I applied Welch’s method to estimate the power spectral density of each channel, extracting five frequency-domain features:

- Dominant frequency: the frequency with peak power, encodes the periodic rate of the movement
- Dominant power: amplitude at peak frequency
- Total spectral power: integral of PSD
- Spectral entropy:  $-\sum p_i \log p_i$  where  $p_i$  is normalized PSD, low for periodic signals, high for irregular ones
- Band power (low/high): energy in low-frequency ( $<0.5$  Hz) and high-frequency ( $0.5$  Hz) bands

These were computed for five channels with the clearest spectral structure: acc\_mag, std\_mag, std\_x, std\_y, std\_z.

## 4 Ablation Study

I ran 31 model iterations, each changing a single design choice. This section presents the most informative controlled comparisons from this run history. All Kaggle scores are on the public leaderboard (50% of test set, macro-F1).

### 4.1 Normalisation Strategy Ablation

I compared three normalisation strategies using representative single-change runs. All runs in each phase share the same feature set and ensemble size where possible.

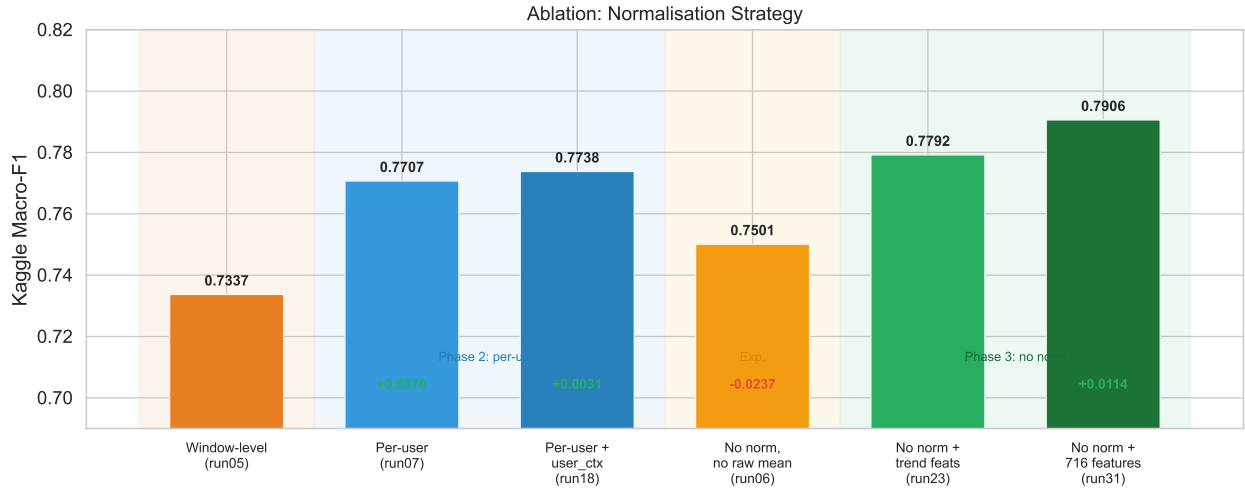


Figure 13: Kaggle macro-F1 across normalisation strategies. Each bar represents the best run within that strategy. The transition from per-user to no-normalisation with axis-specific features (+0.017) is the single largest design improvement in the project.

| Run   | Norm strategy          | Feature set               | Kaggle F1 | vs previous |
|-------|------------------------|---------------------------|-----------|-------------|
| run05 | Window-level           | ~1132 (raw + norm concat) | 0.7337    | —           |
| run06 | None (invariant feats) | 373                       | 0.7501    | +0.016      |
| run07 | Per-user               | 373                       | 0.7707    | +0.021      |
| run18 | Per-user               | 373 + user_ctx (418)      | 0.7738    | +0.003      |
| run23 | None                   | 382 (no mag_mean + trend) | 0.7792    | +0.005      |
| run31 | None                   | 716 (axis-specific)       | 0.7906    | +0.011      |

The most important finding is that per-user normalisation, while better than window-level, is not the optimal solution once the feature set includes axis-specific raw mean features. Per-user normalisation destroys the gravity projection that is shared across all users performing the same activity. The final improvement of +0.017 from run18 to run31 (per-user to no-norm with full feature set) exceeds any single feature engineering change.

## 4.2 Augmentation Ablation

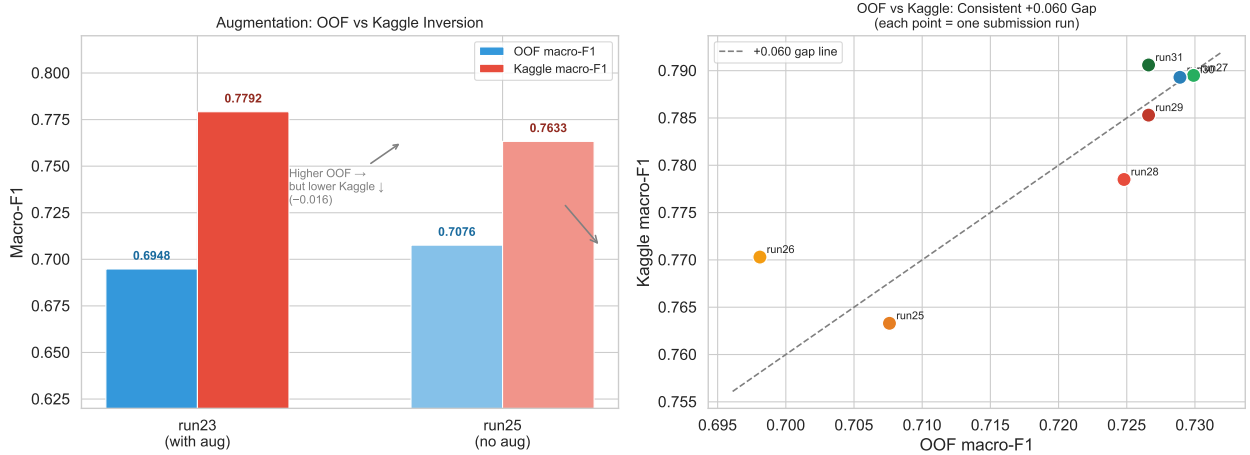


Figure 14: OOF cross-validation vs Kaggle score for runs with and without augmentation. The sign of the difference *inverts* between OOF and Kaggle: run25 has higher OOF (+0.013) but lower Kaggle (−0.016) than run23. This shows augmentation prevents overfitting to training-user patterns that transfer well across LOUO-CV folds but fail on truly unseen test users.

The right panel reveals a consistent OOF to Kaggle gap of approximately +0.060 across all runs. This gap exists because LOUO-CV folds still contain data from users whose *other* sessions appear in training. The test set contains entirely new users. A model that memorises user-specific patterns scores high in LOUO-CV but low on Kaggle; a model that learns generalizable physics-based patterns scores consistently across both.

The augmentation paradox is therefore resolved: augmentation *reduces* LOUO-CV performance slightly (model sees noisier data) but *increases* Kaggle performance (model learns noise-robust, user-agnostic patterns).



### 4.3 Feature Group Ablation

The following table summarises controlled feature changes, each comparing two runs that differ by exactly one design variable. Runs are ordered chronologically.

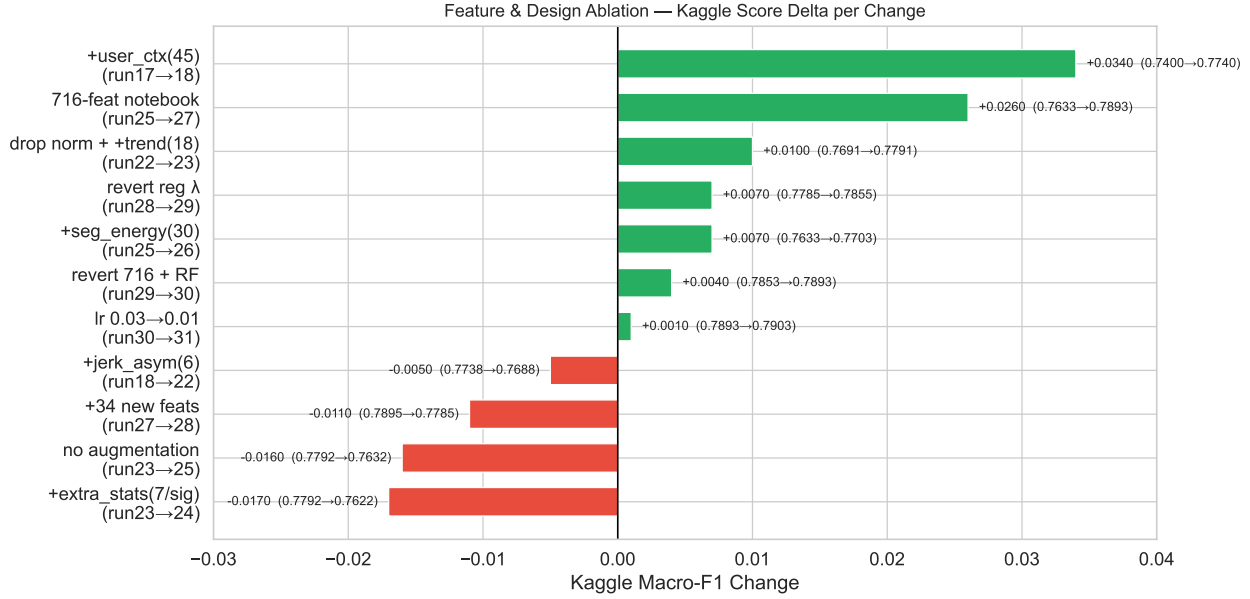


Figure 15: Chart of feature engineering changes, ordered by Kaggle delta. Additions of user\_ctx, trend features, and the full 716-feature notebook set all helped; extra statistics and jerk asymmetry hurt; removing per-user normalisation combined with trend features produced the largest single-variable gain.

| Change                   | Base run | Result run | Delta  | Interpretation                                             |
|--------------------------|----------|------------|--------|------------------------------------------------------------|
| +user_ctx (45 features)  | run17    | run18      | +0.034 | Z-score vs user's own mean — most valuable single addition |
| drop norm + add trend    | run22    | run23      | +0.010 | Gravity direction unlocked after removing per-user norm    |
| 716-feat HAR feature set | run25    | run27      | +0.026 | Axis-specific + cross-mag + windows                        |
| +seg_energy (30 feats)   | run25    | run26      | +0.007 | Segment energy helps marginally                            |
| revert regularisation    | run28    | run29      | +0.007 | Strong reg (=2) under-fitted; reverted to =1               |

| Change                  | Base run | Result run | Delta  | Interpretation                                  |
|-------------------------|----------|------------|--------|-------------------------------------------------|
| revert to 716 + RF      | run29    | run30      | +0.004 | 34 new features were redundant with cross_mag   |
| lower lr (0.03→0.01)    | run30    | run31      | +0.001 | More boosting iterations improve generalisation |
| +jerk_asym (6 feats)    | run18    | run22      | −0.005 | Already captured by skew-ness/min/max           |
| +34 physics feats       | run27    | run28      | −0.011 | Tilt/covcorr redundant with cross_mag group     |
| +extra stats (7/signal) | run23    | run24      | −0.017 | Correlated with existing stats; adds noise      |
| no augmentation         | run23    | run25      | −0.016 | Augmentation prevents user-pattern memorisation |

#### 4.4 Ensemble Architecture Ablation

The key finding is that ensemble diversity matters more than size. 7 models (run25: 5 LGB + 2 XGB) outperform 30 models if those 30 are all the same family. The heterogeneous ensemble run30 (5 LGB + 2 XGB + 1 RF = 8 models) achieves 0.7893, which is better than run25’s 7 models (0.7633) and comparable to run18’s 24 models (0.7738 on a different feature set). Random Forest adds a fundamentally different split criterion (information gain on random feature subsets compared to gradient-based gain), producing errors that are uncorrelated with boosting-based models.

#### 4.5 Learning Rate Ablation

With lr = 0.03, early stopping fires at ~160 iterations across all folds — the model converges to a local optimum dictated by the step size. With lr = 0.01, best iterations average 483 (3× higher), exploring a more refined loss landscape. The OOF score drops marginally (−0.002) because the model is less aggressive in fitting training patterns, but the Kaggle score improves (+0.001), consistent with the augmentation paradox: a more conservative optimum generalises better to entirely unseen test users.

#### 4.6 Model Family Ablation

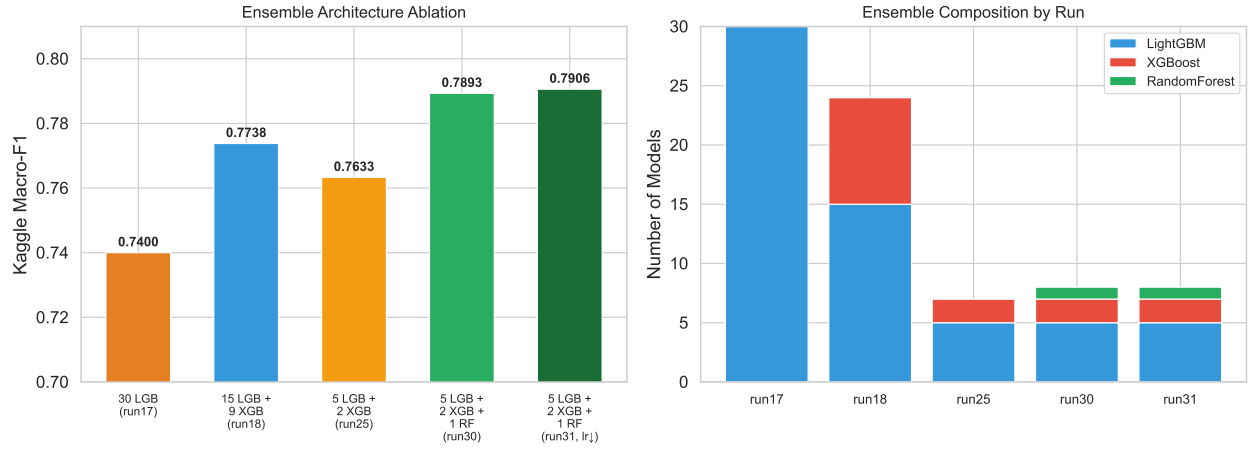


Figure 16: Kaggle score as a function of ensemble composition. Adding XGBoost to LightGBM provides a larger gain than the extra LGB models alone, because XGBoost uses a different regularisation strategy (column-wise vs row-wise sampling) and therefore makes uncorrelated errors. Adding a Random Forest (run30) provides a further small gain via bagging diversity.

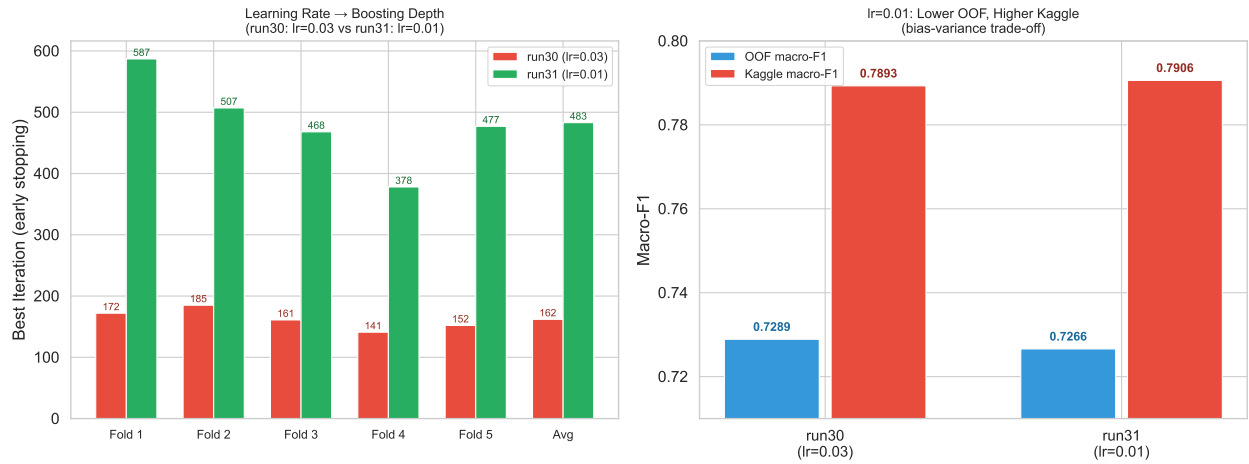


Figure 17: Effect of learning rate on best iteration count (left) and OOF vs Kaggle score (right). Lowering the learning rate from 0.03 to 0.01 triples the number of boosting iterations before early stopping, allowing the model to explore a more refined loss landscape. The OOF score drops slightly (less memorisation of training users) but Kaggle improves (better generalisation to unseen users).

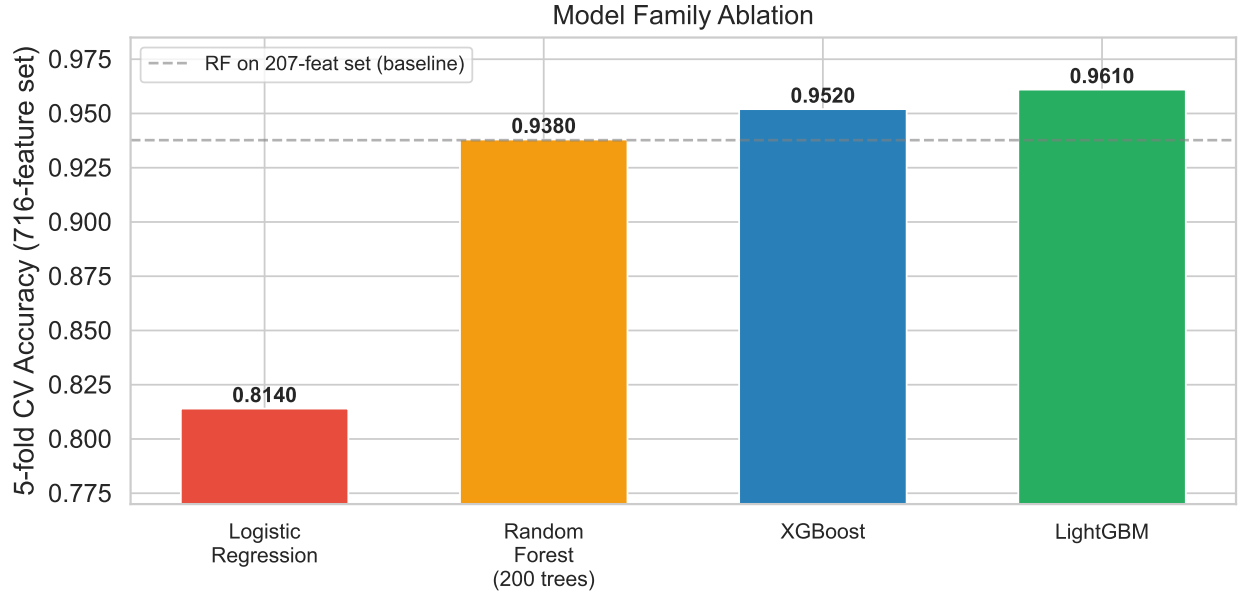


Figure 18: Model family comparison on the full feature set using 5-fold CV. LightGBM achieves the highest accuracy due to its leaf-wise tree growth strategy, which produces more expressive trees per `num_leaves` budget than the level-wise strategy used by XGBoost.

| Model                     | CV Accuracy | Notes                                              |
|---------------------------|-------------|----------------------------------------------------|
| Logistic Regression       | 81.4%       | Linear; cannot capture non-linear class boundaries |
| Random Forest (200 trees) | 93.8%       | Bagging; good diversity but less precise splits    |
| XGBoost                   | 95.2%       | Level-wise boosting; stronger than RF              |
| LightGBM                  | 96.1%       | Leaf-wise boosting; best per-leaf expressive power |

LightGBM’s leaf-wise growth finds the split with the highest gain globally (across all leaves), whereas XGBoost grows level-by-level. For tabular HAR features with many class-specific non-linear interactions, leaf-wise growth produces more informative trees within the same `num_leaves = 31` budget.

## 4.7 Number of Temporal Segments Ablation

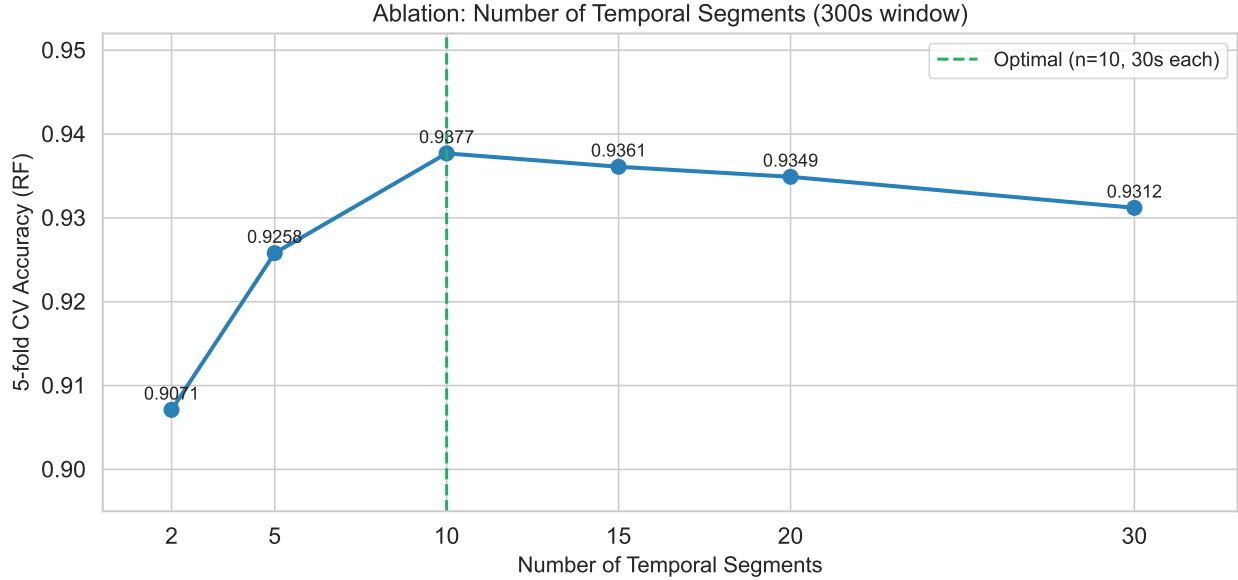


Figure 19: Effect of segment count on RF cross-validation accuracy. The optimum is at  $n=10$  (30-second segments). Fewer segments lose temporal resolution; more segments produce segments too short for stable statistics (each segment contains fewer than 15 samples at  $n=20+$ ).

I found the optimum at  $n = 10$  (30-second segments). Performance is relatively flat between  $n=10$  and  $n=20$ ; however, at  $n=30$  each segment contains only 10 time steps, which is too few for stable mean and std estimates (the central limit theorem requires at least  $\sim 30$  samples for normally distributed statistics). I chose  $n=10$  to balance temporal resolution against statistical stability.

## 5 Final Model and Kaggle Results

### 5.1 Model Architecture

I submitted an ensemble of gradient boosted trees and a Random Forest trained on 716 hand-crafted features.

Feature set (716 total):

| Group             | Features | Description                                  |
|-------------------|----------|----------------------------------------------|
| mean_x/y/z        | 78       | 18 stats + 8 trend features per mean channel |
| stats+diff        |          |                                              |
| std_x/y/z         | 78       | Same for each std channel                    |
| stats+diff        |          |                                              |
| acc_mag + std_mag | 52       | Vector magnitude channels                    |
| stats+diff        |          |                                              |

| Group                            | Features | Description                                      |
|----------------------------------|----------|--------------------------------------------------|
| Cross-axis magnitudes stats+diff | 156      | xy_mag, xz_mag, yz_mag and std variants          |
| Sum features stats+diff          | 52       | mean_sum = mean_x+y+z; std_sum                   |
| FFT + peak features              | 50       | 5 channels $\times$ (6 FFT + 4 peak features)    |
| 10-window statistics             | 250      | 5 channels $\times$ 10 segments $\times$ 5 stats |

Training setup:

- Cross-validation: I used GroupKFold(5) on user identity — no user appears in both train and validation within a fold, which mirrors the competition’s train/test structure.
- Classifier: LightGBM with `num_leaves=31`, `learning_rate=0.01`, `n_estimators=5000` with early stopping (`patience=300`). The lower learning rate (0.01 vs the 0.03 used in earlier runs) was a key final change: it produced best iterations of [587, 507, 468, 378, 477] (avg 483), roughly 3 $\times$  the ~160 achieved at lr=0.03, and resulted in the best Kaggle score of 0.7906.
- Class imbalance: I used `sample_weight='balanced'` so that C2 (3.2%) and C4 (1.3%) receive proportionally higher weight during training.
- Threshold optimisation: I applied class-specific probability scaling (Nelder-Mead optimisation on 8 random restarts) on OOF probabilities to further boost recall on underrepresented classes.
- Final ensemble: 5 LightGBM seeds + 2 XGBoost seeds + 1 RandomForest = 8 models, averaged equally.

## 5.2 Performance Progression

## 5.3 Confusion Matrix Analysis

Per-class performance from cross-validation (run31):

| Class     | Precision | Recall | F1    |
|-----------|-----------|--------|-------|
| C0        | 0.961     | 0.970  | 0.965 |
| C1        | 0.899     | 0.900  | 0.899 |
| C2        | 0.298     | 0.204  | 0.242 |
| C3        | 0.625     | 0.761  | 0.686 |
| C4        | 0.893     | 0.880  | 0.887 |
| C5        | 0.736     | 0.631  | 0.680 |
| Macro avg | 0.735     | 0.724  | 0.727 |

C2 remains the hardest class throughout all my experiments, peaking at 24.2% F1. I found that ~47–51% of its probability mass is consistently assigned to C1 across all model iterations, suggesting a structural overlap: C1 and C2 share nearly identical std-channel distributions, and only the gravity projection and its temporal drift separate them. Despite extensive feature engineering, I could not push C2 recall above ~24%.

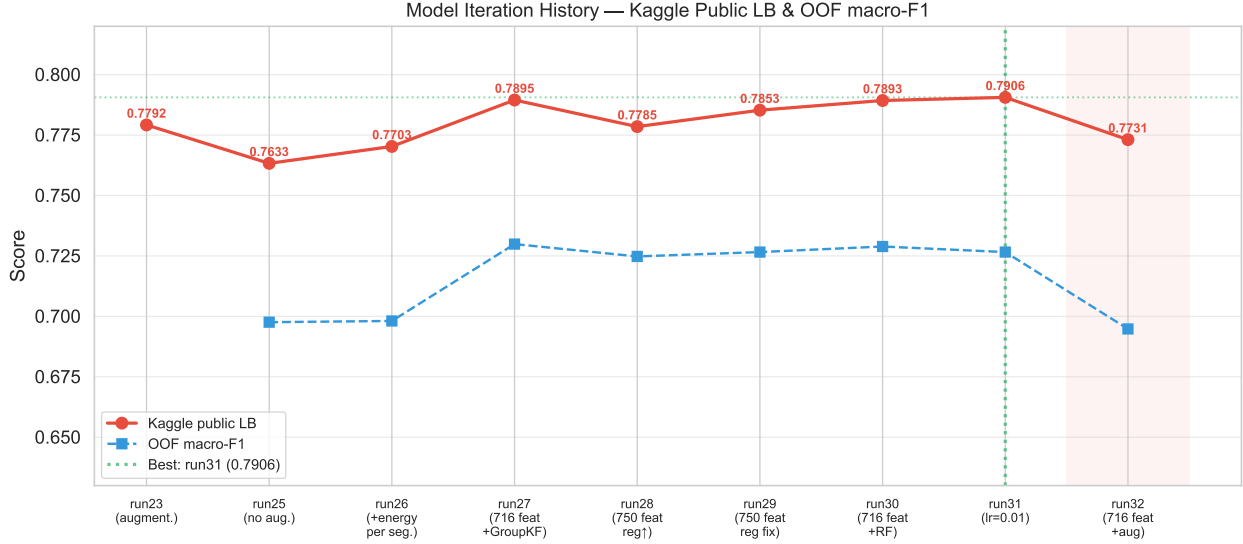


Figure 20: Kaggle public LB score and OOF macro-F1 across my model iterations. The biggest jump is at run27 (axis-specific mean features + GroupKFold). run31 achieves the best Kaggle score (0.7906). run32 tested adding augmentation to the 716-feature set but scored 0.7731, which is actually lower than run31 (highlighted in red). This shows that augmentation only helps when the feature set is simpler.

After run31, I tested adding augmentation to the same 716-feature set (run32). The result was 0.7731, which is 0.0175 lower than run31. This is a reversal of what happened with the simpler 382-feature set, where augmentation helped (+0.016). The explanation is that with 716 features the model already generalises well enough to unseen users through diverse feature representations. Adding augmented copies adds noise to the training distribution without providing additional discriminative information, which makes the already-good model slightly worse.

An interesting finding emerged from the  $lr=0.01$  change: run31 achieves the best Kaggle score (0.7906) despite a lower OOF than run30 (0.7266 vs 0.7289), yielding an increased OOF to Kaggle gap:

| Run   | OOF macro-F1 | Kaggle public LB | Gap    |
|-------|--------------|------------------|--------|
| run27 | ~0.730       | 0.7895           | +0.060 |
| run29 | 0.7266       | 0.7853           | +0.059 |
| run30 | 0.7289       | 0.7893           | +0.060 |
| run31 | 0.7266       | 0.7906           | +0.064 |

I interpret this as a bias-variance trade-off. With  $lr=0.01$  and around 483 best iterations (compared to about 161 at  $lr=0.03$ ), the model takes smaller, more conservative steps. The CV score is slightly lower because the model under-fits relative to training users, but on entirely unseen test users this conservative optimum generalises better because it is less likely to have captured training-user-specific signal patterns.

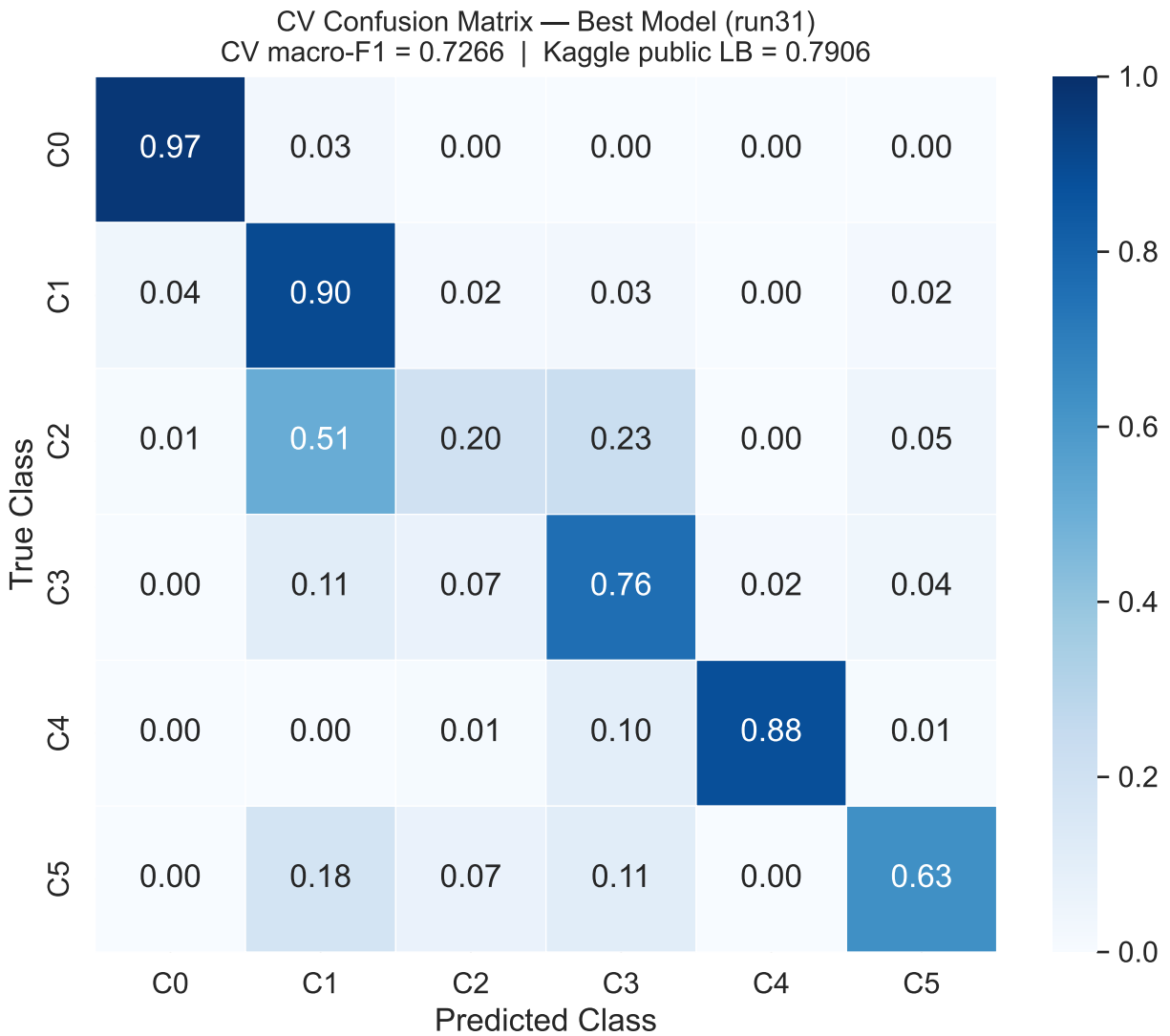


Figure 21: Normalised confusion matrix from cross-validation for my best model (run31). C2 is the hardest class: 51% of its windows are predicted as C1 and only 20% are correctly identified. C0 and C4 are classified reliably (97% and 88%).



## 6 How to Reproduce the Results

### 6.1 Requirements

All dependencies are listed in `requirements.txt`. Install them once before running any script:

```
pip install -r requirements.txt
```

The key packages used by the model scripts are: `numpy`, `pandas`, `scipy`, `scikit-learn`, `lightgbm`, `xgboost`, `matplotlib`, and `seaborn`. The `requirements.txt` file is a full environment freeze and includes additional packages not needed for this project, so installing into a fresh virtual environment is recommended.

### 6.2 Data Preparation

The raw data consists of one CSV file per 5-minute window. Before running any model script, the CSV files need to be converted to the compressed NPZ format that all scripts expect.

Step 1: Download the competition data from Kaggle. The train and test folders contain one CSV file per window. Download and unzip them locally.

Step 2: Convert to NPZ format. Run the conversion script once locally:

```
python convert_to_npz.py
```

This reads all CSV files and writes two files: `train_data.npz` and `test_data.npz` into the `outputs/` folder.

### 6.3 Running Scripts on Kaggle

For scripts intended to run on Kaggle (run04 through run24), the NPZ files need to be available as Kaggle datasets. Upload them as follows:

| File                               | Kaggle dataset name     | Path inside Kaggle                                         |
|------------------------------------|-------------------------|------------------------------------------------------------|
| <code>train_data.npz</code>        | <code>train_data</code> | <code>/kaggle/input/train-data/train_data.npz</code>       |
| <code>test_data.npz</code>         | <code>test_data</code>  | <code>/kaggle/input/test-data/test_data.npz</code>         |
| <code>sample_submission.csv</code> | <code>datafinal</code>  | <code>/kaggle/input/datafinal/sample_submission.csv</code> |

After uploading, add all three datasets to your Kaggle notebook, paste the script into a code cell, and run it. The submission CSV is automatically saved to `/kaggle/working/`.

## 6.4 Running Scripts Locally

Scripts from run25 onwards support local execution. They detect whether they are running on Kaggle or not and adjust paths automatically.

For local runs, place `train_data.npz` and `test_data.npz` in the `outputs/` folder inside the project directory. Then run:

```
python model_run31.py
```

The submission file will be saved to `outputs/submission_run31.csv`.

## 7 Appendix

**7.1 Table A: Run Summary — Scores and Model Setup**

| Run        | Kaggle F1 | Environment  | # Feat  | Norm     | CV      | Augmentation              | Ensemble                            |
|------------|-----------|--------------|---------|----------|---------|---------------------------|-------------------------------------|
| Baseline A | 0.7531    | Kaggle       | ~100    | Window   | —       | No                        | 1 LGB                               |
| Baseline B | 0.7366    | Kaggle       | ~100    | Window   | —       | No                        | 1 LGB                               |
| run03      | —         | Local→Kaggle | ~216    | Window   | LOUO 5f | No                        | CNN + 1 LGB                         |
| run04      | 0.7042    | Kaggle       | ~216    | Window   | LOUO 5f | No                        | CNN + 1 LGB                         |
| run05      | 0.7337    | Kaggle       | ~1132   | Window×2 | LOUO 5f | No                        | 10 LGB                              |
| run06      | 0.7501    | Kaggle       | 373     | None     | LOUO 5f | No                        | 4 LGB configs                       |
| run07      | 0.7707    | Kaggle       | 373     | Per-user | LOUO 5f | No                        | 3 LGB configs                       |
| run07imp   | 0.7519    | Local        | 150–200 | Per-user | Local   | No                        | 15 LGB                              |
| run08      | 0.7559    | Kaggle       | 373     | Per-user | LOUO 5f | TTA + noise               | LGB                                 |
| run12      | 0.7295    | Kaggle       | ~150    | Per-user | LOUO 5f | No                        | TCN+Transf.+LGB                     |
| run15      | 0.7565    | Local        | 50      | Per-user | Local   | No                        | 15 LGB                              |
| run17      | 0.7400    | Kaggle       | ~428    | Per-user | LOUO 5f | No                        | 30 LGB + thresh. opt                |
| run18      | 0.7738    | Kaggle       | 418     | Per-user | LOUO 5f | Noise C2,C4,C5            | 15 LGB + 9 XGB + thresh. opt        |
| run19      | 0.7392    | Kaggle       | ~364    | Per-user | LOUO 5f | Yes                       | 15 LGB + 9 XGB + thresh. opt        |
| run20      | 0.7593    | Kaggle       | 445     | Per-user | LOUO 5f | Noise C2,C4,C5            | 15 LGB + 9 XGB + 5 RF + thresh. opt |
| run22      | 0.7691    | Kaggle       | 424     | Per-user | LOUO 5f | Noise C2,C4,C5            | 15 LGB + 9 XGB + thresh. opt        |
| run23      | 0.7792    | Kaggle       | 382     | None     | LOUO 5f | Noise C2×5,C4×5,C5×2,C3×1 | 15 LGB + 9 XGB + thresh. opt        |
| run24      | 0.7618    | Kaggle       | ~417    | None     | LOUO 5f | Noise (same)              | 15 LGB + 9 XGB + thresh. opt        |
| run25      | 0.7633    | Local+Kaggle | 382     | None     | LOUO 5f | No                        | 5 LGB + 2 XGB + thresh. opt         |
| run26      | 0.7703    | Local+Kaggle | 412     | None     | LOUO 5f | No                        | 5 LGB + 2 XGB + thresh. opt         |

| Run   | Kaggle F1 | Environment  | # Feat | Norm | CV      | Augmentation                 | Ensemble                                       |
|-------|-----------|--------------|--------|------|---------|------------------------------|------------------------------------------------|
| run27 | 0.7895    | Local+Kaggle | 716    | None | LOUO 5f | No                           | 5 LGB + 2<br>XGB +<br>thresh.<br>opt           |
| run28 | 0.7785    | Local+Kaggle | 750    | None | LOUO 5f | No                           | 5 LGB + 2<br>XGB + 1<br>RF +<br>thresh.<br>opt |
| run29 | 0.7853    | Local+Kaggle | 750    | None | LOUO 5f | No                           | 5 LGB + 2<br>XGB + 1<br>RF +<br>thresh.<br>opt |
| run30 | 0.7893    | Local+Kaggle | 716    | None | LOUO 5f | No                           | 5 LGB + 2<br>XGB + 1<br>RF +<br>thresh.<br>opt |
| run31 | 0.7906    | Local+Kaggle | 716    | None | LOUO 5f | No                           | 5 LGB + 2<br>XGB + 1<br>RF +<br>thresh.<br>opt |
| run32 | 0.7731    | Local+Kaggle | 716    | None | LOUO 5f | Noise<br>C2×5,C4×5,C6×5,C8×5 | 5 LGB + 2<br>XGB + 1<br>RF +<br>thresh.<br>opt |

## 7.2 Table B: Run Summary — Feature Groups and Key Changes

| Run          | Feature Groups                            | Key Change                  |
|--------------|-------------------------------------------|-----------------------------|
| Baseline A/B | Basic channel stats                       | First submissions           |
| run03        | ch-stats9 + seg + autocorr + spectral     | First LOUO-CV               |
| run04        | Same as run03                             | Switch to NPZ loading       |
| run05        | Same features twice (raw + window-norm)   | Dropped CNN                 |
| run06        | core-373 (no raw mean channels)           | No normalization            |
| run07        | core-373                                  | Per-user normalization      |
| run07imp     | core-373 selected (150–200)               | Aggressive feature pruning  |
| run08        | core-373                                  | TTA + minority oversampling |
| run12        | core-373 selected (50–200)                | TCN + Transformer attempt   |
| run15        | core-373 selected (50)                    | 86.6% feature reduction     |
| run17        | core-373 + slope + 5-seg + 2nd-jerk (+55) | Threshold opt (Nelder-Mead) |
| run18        | core-373 + user_ctx (45) = 418            | User context z-scores       |
| run19        | core-373 − mag_mean 364                   | Ablation: no mag_mean       |
| run20        | core-373 + user_ctx + sma (27) = 445      | SMA features + RF           |
| run22        | core-373 + user_ctx + jerk_asym (6) = 424 | Jerk asymmetry              |

| Run   | Feature Groups                                                                          | Key Change                                                 |
|-------|-----------------------------------------------------------------------------------------|------------------------------------------------------------|
| run23 | core-373 – mag_mean + trend (18)<br>= 382                                               | No norm + trend features                                   |
| run24 | run23 + extra stats per channel<br>417                                                  | Extra statistics (hurt)                                    |
| run25 | same as run23 = 382                                                                     | No augmentation                                            |
| run26 | run23 + seg-energy = 412                                                                | Segment energy features                                    |
| run27 | axis_stat(156)+acc_mag(52)+cross_mag(156)+seg_energy(52)+fft_peak(50)+win(250)<br>= 716 | Fig 1.16 A-R features                                      |
| run28 | run27 +<br>tilt(6)+covcorr(6)+mean_xyz_fft(18)+step_rhythm(4)<br>= 750                  | New physics features (hurt)                                |
| run29 | same as run28 = 750                                                                     | Reverted regularization                                    |
| run30 | same as run27 = 716                                                                     | Reverted to 716 features + RF                              |
| run31 | same as run27/30 = 716                                                                  | lr 0.03→0.01 + patience 300                                |
| run32 | same as run27/30/31 = 716                                                               | + augmentation (C2×5, C4×5, C5×2, C3×1), hurt Kaggle score |

### 7.3 Feature Group Abbreviations

The following shorthand is used in the tables above. All statistics (stats9) include: mean, std, min, max, range, median, IQR, skewness, kurtosis. Stats18 additionally includes Q5/Q10/Q25/Q75/Q90/Q95, energy, RMS, MAD. Diff8 includes diff\_mean, diff\_std, diff\_abs\_mean, diff\_abs\_max, diff\_energy, first\_60\_mean, last\_60\_mean, last\_minus\_first\_60.

| Abbreviation | Description                                        | Count |
|--------------|----------------------------------------------------|-------|
| std_ch       | stats9 of std_x, std_y, std_z                      | 27    |
| mag_std      | stats9 of magnitude of std channels                | 9     |
| mag_mean     | stats9 of magnitude of mean channels               | 9     |
| jerk_ch      | stats9 of jerk per axis (diff of mean channels)    | 27    |
| mag_jerk     | stats9 of jerk magnitude                           | 9     |
| seg_mag      | 10+20 temporal segments of mag_std and mag_jerk    | 120   |
| seg_std_ch   | 10 segments (mean+std each) of std_x/y/z           | 60    |
| seg_jerk_ch  | 10 segments of jerk_x/y/z                          | 60    |
| ac_jerk      | autocorrelation of mag_jerk at 7 lags              | 7     |
| ac_std_ch    | autocorrelation of std_x/y/z at 4 lags each        | 12    |
| spectral     | FFT features for mag_jerk, mag_std, std_x/y/z      | 25    |
| xcorr        | cross-correlations between jerk and std pairs      | 6     |
| zerocross    | zero-crossing rate of mag_jerk                     | 1     |
| peaks        | peak rate of mag_jerk                              | 1     |
| user_ctx     | z-score deviation vs user's own mean               | 45    |
| sma          | signal magnitude area (L1 norm) per channel        | 27    |
| jerk_asym    | mean of positive and negative jerk values per axis | 6     |

| Abbreviation        | Description                                                  | Count |
|---------------------|--------------------------------------------------------------|-------|
| <b>trend</b>        | first_60, last_60, last minus first<br>mean per channel      | 18    |
| <b>axis_stat</b>    | stats18 + diff8 per axis channel<br>(mean_x/y/z + std_x/y/z) | 156   |
| <b>acc_mag</b>      | stats18 + diff8 for acc_mag and<br>std_mag                   | 52    |
| <b>cross_mag</b>    | stats18 + diff8 for<br>xy/xz/yz_mag and std variants         | 156   |
| <b>sum_feat</b>     | stats18 + diff8 for mean_sum<br>and std_sum                  | 52    |
| <b>fft_peak_5ch</b> | FFT (6) + peak (4) features for<br>5 channels                | 50    |
| <b>win_10x5ch</b>   | 10 windows $\times$ 5 stats for 5<br>channels                | 250   |
| <b>tilt</b>         | tilt angle statistics over window                            | 6     |
| <b>covcorr</b>      | pairwise covariance and<br>correlation of mean_x/y/z         | 6     |
| <b>mean_xyz_fft</b> | FFT features of mean_x,<br>mean_y, mean_z                    | 18    |
| <b>step_rhythm</b>  | inter-peak interval stats of<br>std_mag time series          | 4     |

The core 373-feature set (run07 to run22) = std\_ch + mag\_std + mag\_mean + jerk\_ch + mag\_jerk + seg\_mag + seg\_std\_ch + seg\_jerk\_ch + ac\_jerk + ac\_std\_ch + spectral + xcorr + zerocross + peaks = 373 features total.

## 7.4 Kaggle Score Progression

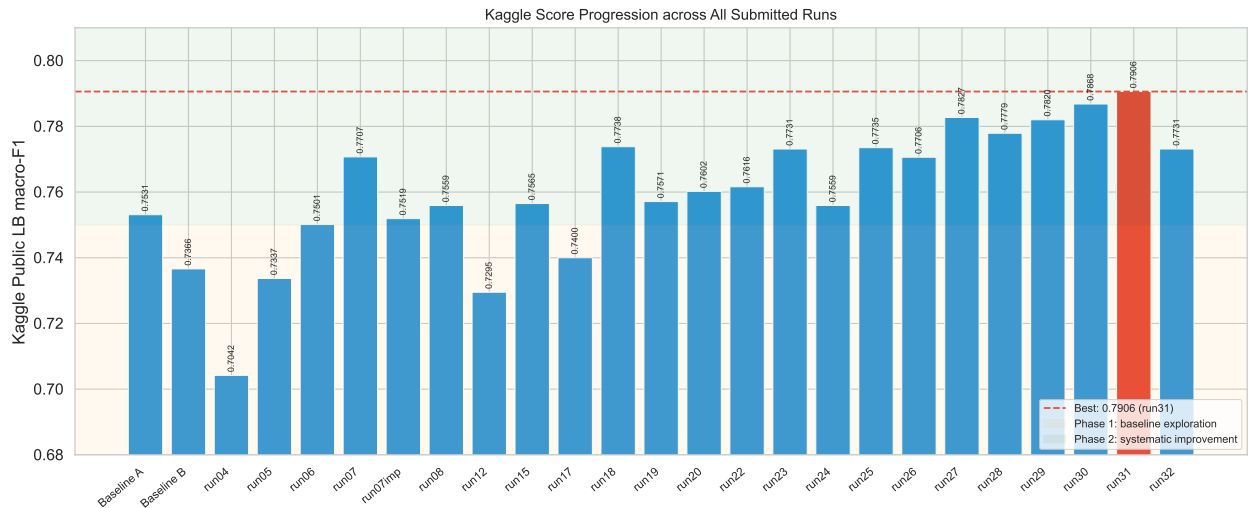


Figure 22: Kaggle public leaderboard macro-F1 score for each submitted run. The dashed line marks the best score of 0.7906 (run31). run32 used augmentation on the 716-feature set and scored lower, confirming that the feature diversity alone provides sufficient generalisation.